Programação Orientada a Objetos

Aula 02: Introdução à Programação em Java

jshell e elementos de um programa (tipos primitivos, operadores, keywords, variáveis e etc)

Prof. Fabricio Santana

2025.1

fabricio.santana@idp.edu.br

www.linkedin.com/in/fabriciofsantana

O que é programação?

conjunto de instruções

"Programação é a arte de dizer ao computador o que fazer"

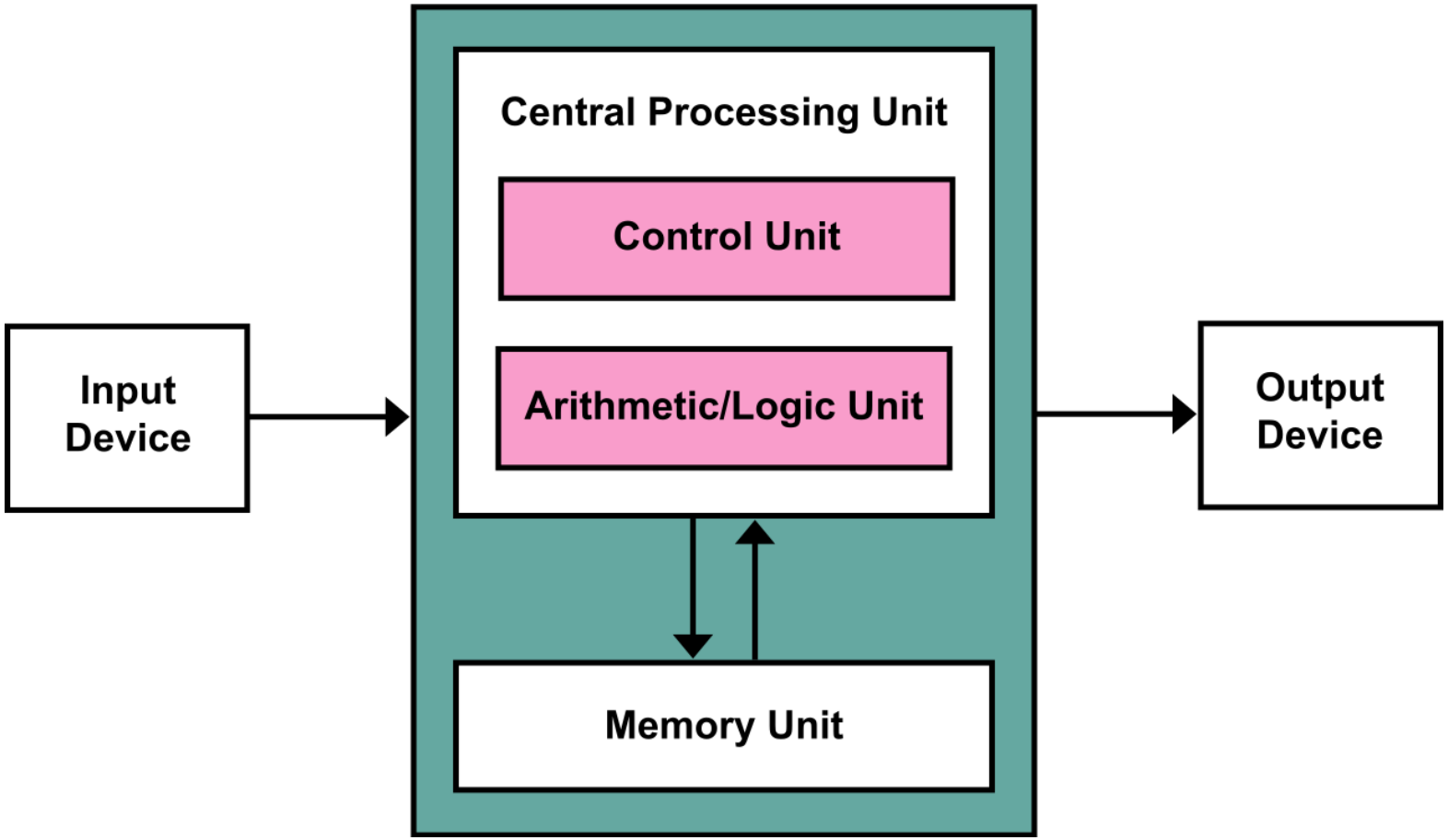
tarefas

(Donal Knuth, autor da série de livros *The Art of Computer Programming*)

"Um algoritmo é um **conjunto de instruções** para concluir uma **tarefa**"

Fonte: BHARGAVA, Aditya Y. Grokking Algorithms. 2. ed. Shelter Island: Manning Publications, 2024. ISBN 978-1-63343-853-8.

Como o computador executa a tarefa?



Fonte: https://en.wikipedia.org/wiki/Von_Neumann_architecture

Qual a função de um programa?

- Manipular dados

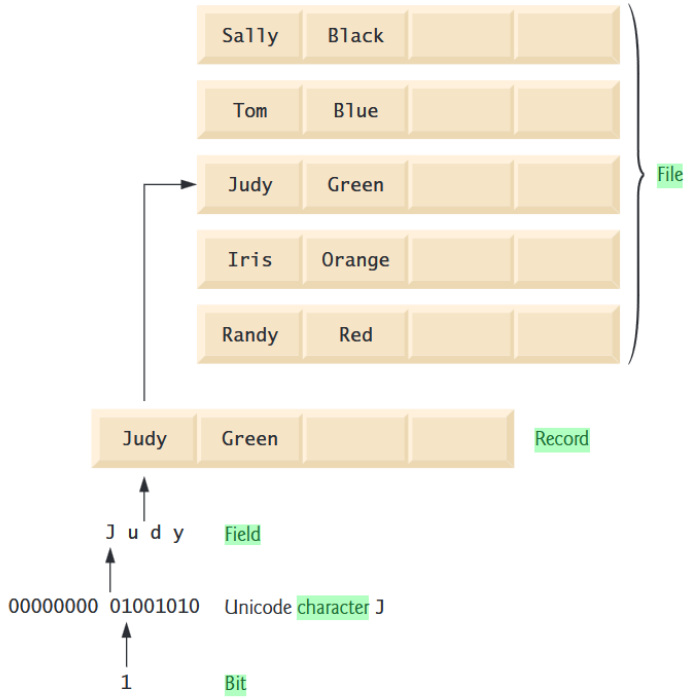


Fig. 1.3 | Data hierarchy.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Qual a função de um programa?

- Manipular dados

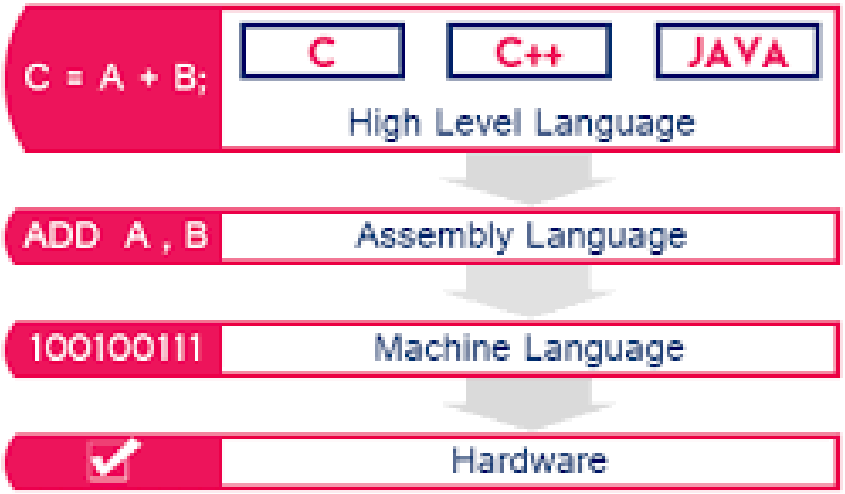
Unit	Bytes	Which is approximately
1 kilobyte (KB)	1024 bytes	10^3 (1024) bytes exactly
1 megabyte (MB)	1024 kilobytes	10^6 (1,000,000) bytes
1 gigabyte (GB)	1024 megabytes	10^9 (1,000,000,000) bytes
1 terabyte (TB)	1024 gigabytes	10^{12} (1,000,000,000,000) bytes
1 petabyte (PB)	1024 terabytes	10^{15} (1,000,000,000,000,000) bytes
1 exabyte (EB)	1024 petabytes	10^{18} (1,000,000,000,000,000,000) bytes
1 zettabyte (ZB)	1024 exabytes	10^{21} (1,000,000,000,000,000,000,000) bytes

Fig. 1.4 | Byte measurements.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Como “instruir” um computador?

- Linguagem de programação
  - Linguagem de máquina
  - Linguagem assembly
  - Linguagem de alto nível
    - Ex.: Java



Fonte: http://www.btechsmartclass.com/c_programming/C-Computer-Languages.html

Qual o próximo passo?

Fonte: https://www.youtube.com/watch?v=8LmfkUb2uIY

- Progresso dos modelos da openAI em codificação:
  - 1º modelo de raciocínio → top 1 milhão
  - o1 (setembro/2024) → top 10.000
  - o3 (janeiro/2025) → top 175
  - modelo interno (fevereiro/2025) → top 50
  - Final de 2025 → top 1

O que usar para programar em Java?

- Ferramentas (*Java Development Kit*)
  - *jshell*: read-eval-print-loop (*REPL*)
  - *javac*: compila código *java* em *byte code*
  - *java:* executa uma aplicação java
  - *jar: compacta os arquivos para distribuição*
  - *javadoc:* gera documentação do código
  - *jpackage:* para distribuição como um executável
  - ...
  - ...
  - ...

Fonte: https://docs.oracle.com/en/java/javase/21/docs/specs/man/index.html

jshell

- *$ jshell*

```
@fabriciosantana ➜/workspaces/poo (2025.1) $ jshell
|  Welcome to JShell -- Version 21.0.2
|  For an introduction type: /help intro

jshell>
```

- *jshell> System.out.println("Hello World!")*

```
@fabriciosantana ➜/workspaces/poo (2025.1) $ jshell
|  Welcome to JShell -- Version 21.0.2
|  For an introduction type: /help intro

jshell> System.out.println("Hello World!")
Hello World!

jshell>
```

Jshell commands

- *jshell>/list*
- *jshell>/edit [option]*
- *jshell>/drop {name|id|startID-endID}*
- *jshell>/save [options] file*
- *jshell>/open file*
- *jshell>/reload*
- *jshell>/reset*
- *jshell>/history*
- *jshell>/types [option]*
- *jshell>/vars*
- *jshell>/help*

Fonte: https://docs.oracle.com/en/java/javase/21/docs/specs/man/jshell.html

Elementos de um programa Java

- *literals*
- *identifiers*
- *variables*
- *data types*
- *keywords*
- *operators*
- *separators*

- *declarations*
- *expressions*
- *statements*

Fonte: https://docs.oracle.com/javase/specs/jls/se21/html/index.html

Tipos primitivos

- Os tipos primitivos são os tipos de dados mais básicos
- Java possui 8 tipos primitivos

Números inteiros (*Integers*)	Números reais ou de ponto flutuante (*floating-point numbers*)
byte short **int** long	float **double**
Caractere	**Valor booleano**
char	boolean

- Java possui também tipos “especiais” (classes): reference types
  - built-in: disponíveis por padrão no Java
  - Custom: desenvolvido

- *Primitive Types x Reference Types*

Tipos Primitivos

- *jshell> 1+1;*
- *jshell> int number1;*
- *jshell> int number2;*
- *jshell> number1 = 10;*
- *jshell> number1 = 5;*
- *jshell> sum = number1 + number2;*
- *jshell> int newSum = sum + 3;*
- *jshell> int x =(10 + 5) + (2 * 10);*
- *jshell> int y = 12;*
- *jshell> int z = x + y;*
- *jshell> z;*

Tipos Primitivos

- jshell> {
```
int a = 2;
int b = 2;
int c = a * b;
System.out.println(c);
}
```
- jshell> {
```
int a = 1;
int b = 2;
int c = a - b;
System.out.println("The difference is: " + c);
}
```

Tipos Primitivos

- *jshell>* {

```
int lightspeed;
long days;
long seconds;
long distance;
// approximate speed of light in miles per second
lightspeed = 186000;
days = 1000; // specify number of days here
seconds = days * 24 * 60 * 60; // convert to seconds
distance = lightspeed * seconds; // compute distance
System.out.print("In " + days);
System.out.print(" days light will travel about ");
System.out.println(distance + " miles.");
}
```

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Tipos Primitivos

- *jshell> double d = 10.25D;*
- *jshell> double d = 10.25;*
- *jshell> float d = 10.25F;*
- *jshell> {*

```
        double pi, r, a;
        r = 10.8; // radius of circle
        pi = 3.1416; // pi, approximately
        a = pi * r * r; // compute area
        System.out.println("Area of circle is " + a);
    }
```

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Tipos Primitivos

- ```
  jshell> {
          char ch1, ch2;
          ch1 = 88; // code for X
          ch2 = 'Y';
          System.out.print("ch1 and ch2: ");
          System.out.println(ch1 + " " + ch2);
  }
  ```

- ```
  jshell> {
          char ch1;
          ch1 = 'X';
          System.out.println("ch1 contains " + ch1);
          ch1++; // increment ch1
          System.out.println("ch1 is now " + ch1);
  }
  ```

- char é armazenado como um número inteiro sem sinal (16 bits) (Ver: unicode.org e symbl.cc)

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Tipos Primitivos

- jshell> {

```
boolean b;
b = false;
System.out.println("b is " + b);
b = true;
System.out.println("b is " + b);
// a boolean value can control the if statement
if(b) System.out.println("This is executed.");
b = false;
if(b) System.out.println("This is not executed.");
// outcome of a relational operator is a boolean value
System.out.println("10 > 9 is " + (10 > 9));
}
```

- jshell> if (denom != 0 && num / denom > 10)

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Tipos Primitivos

- Cada primitivo possui um classe wrapper que fornece informações e operações para sobre o tipo primitivo
- 8 tipos primitivos em Java

Tipo Primitivo	Classe Wrapper
byte	Byte
short	Short
char	Character
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Fonte: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/package-summary.html

Tipos Primitivos

- *jshell> byte minValue = Byte.MIN_VALUE*
- *jshell> byte maxValue = Byte.MAX_VALUE*
- *jshell> short minValue = Short.MIN_VALUE*
- *jshell> short maxValue = Short.MAX_VALUE*
- *jshell> int minValue = Integer.MIN_VALUE*
- *jshell> int maxValue = Integer.MAX_VALUE*
- *jshell> long minValue = Long.MIN_VALUE*
- *jshell> long maxValue = Long.MAX_VALUE*
- *jshell> long longValue = 10L*

- *Overflow e Underflow*

Fonte: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/package-summary.html

Tipos Primitivos

- *jshell> Float.MIN_VALUE*
- *jshell> Float.MAX_VALUE*
- *jshell> Double.MIN_VALUE*
- *jshell> Double.MAX_VALUE*

Fonte: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/package-summary.html

Tamanho dos tipos primitivos

Tipo Primitivo	Tamanho (em bits)	Mínimo	Máximo	Qtd. números
byte	8	-128 (-2^7)	127 (2^7-1)	2^8 = 256
short	16	-32.768 (-2^{15})	32.767 (2^{15}-1)	2^{16} = 65.536
int	32	-2.147.483.648 (-2^{31})	2.147.483.647 (2^{31}-1)	2^{32} = 4.294.967.296
long	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807	2^{64} = 18.446.744.073.709.551.616
float	32	1.40129846432481707e–45	3.4028234663852886E+38	
double	64	4.94065645841246544e–324	1.7976931348623157E+308	
boolean	true or false			
char	16	\u0000	\uFFFF	2^{16} = 65.536

- Como inteiros com sinal são representados no computador? Two's complement
  - Exemplo 8 bits:
    - 5: 00000101
    - -5: 11111011 (inverte-se os bit da representação positiva e soma-se 1)

Literals

- **Integer literals**
  - Decimal → int i = 10;
    - Decimal (64 bits) → long i = 10L;
  - Octal → int i = 010; // 8 na base octal
  - Hexadecimal → int i = 0x10; // 16 na base hexadecimal
  - Binário → int i = 0b10; // 2 na base binária
- **Floating-point literals**
  - doube d = 10.0D; (default, maior precisão)
  - float f = 10.0F;
- **Boolean literals**
  - boolean f = false;
  - boolean t = true;
- **Charecter literals**
  - char c = ‘a’;
  - char c = ‘\141’; // código unicode ‘a’ em octal
  - char c = ‘u\0061’; // código unicode do ‘a’ em hexadecimal

Variables

- type identifier [= value][, identifier [= value] ...];
  - inferência de tipo para variável local (var v = 1;)
- Escopo e inicialização
- *jshell>* *{*

```
int x; // known to all code within main
x = 10;
if(x == 10) { // start new scope
        int y = 20; // known only to this block
        // x and y both known here.
        System.out.println("x and y: " + x + " " + y);
        x = y * 2;
}
// y = 100; // Error! y not known here
// x is still known here.
System.out.println("x is " + x);
}
```

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Variables

- type identifier [= value][, identifier [= value] ...];
  - Toda variável tem um tipo, um nome (identificador), um tamanho (bytes) e um valor
- Escopo e inicialização
- *jshell> {*

```
int x; // known to all code within main
x = 10;
if(x == 10) { // start new scope
        int y = 20; // known only to this block
        // x and y both known here.
        System.out.println("x and y: " + x + " " + y);
        x = y * 2;
}
// y = 100; // Error! y not known here
// x is still known here.
System.out.println("x is " + x);
}
```

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Casting

- *widening conversion*
  - Os tipos são compatíveis
  - O tipo de destino é maior que o tipo da origem

- *narrowing conversion*
  - Tipos incompatíveis, necessário fazer *cast*
    - (target-type) value

- *jshell> byte a = 50; byte b = (byte) (a * 2);*
- *jshell> long l = 10L;*
- *jshell> int i = 5;*
- *jshell> long j = i + l;*
- *jshell> int j = i + (int) l;*
- *jshell> float myFloat = (float) 5.25;*
- *jshell> int i = (int) 10.90;*

Keywords

```
Keyword:
  ReservedKeyword
  ContextualKeyword

ReservedKeyword:
  (one of)

  abstract   continue   for          new         switch
  assert     default    if           package     synchronized
  boolean    do         goto         private     this
  break      double     implements   protected   throw
  byte       else       import       public      throws
  case       enum       instanceof   return      transient
  catch      extends    int          short       try
  char       final      interface    static      void
  class      finally    long         strictfp    volatile
  const      float      native       super       while
  _ (underscore)

ContextualKeyword:
  (one of)

  exports      opens      requires     uses   yield
  module       permits    sealed       var
  non-sealed   provides   to           when
  open         record     transitive   with
```

The keywords const and goto are reserved, even though they are not currently used. This may allow a Java compiler to produce better error messages if these C++ keywords incorrectly appear in programs.

The keyword strictfp is obsolete and should not be used in new code.

The keyword _ (underscore) is reserved for possible future use in parameter declarations.

true and false are not keywords, but rather boolean literals (§3.10.3).

null is not a keyword, but rather the null literal (§3.10.8).

Fonte: https://docs.oracle.com/javase/specs/jls/se21/html/jls-3.html#jls-3.9

Separators

Symbol	Name	Purpose
( )	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[ ]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a **for** statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.
::	Colons	Used to create a method or constructor reference.
...	Ellipsis	Indicates a variable-arity parameter.
@	At-sign	Begins an annotation.

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Operators

- Aritmético

Operator	Result
+	Addition (also unary plus)
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
– =	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement

- Relacional

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

- Lógico

Operator	Result
&	Logical AND
|	Logical OR
^	Logical XOR (exclusive OR)
||	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
|=	OR assignment
^=	XOR assignment

- Bitwise

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
|=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Operators

Operator	Description	Associativity
++ --	unary postfix increment unary postfix decrement	right to left
++ -- + - ! ~ (*type*)	unary prefix increment unary prefix decrement unary plus unary minus unary logical negation unary bitwise complement unary cast	right to left
* / %	multiplication division remainder	left to right
+ -	addition or string concatenation subtraction	left to right
<< >>>>>>	left shift signed right shift unsigned right shift	left to right
< <= >>= instanceof	less than less than or equal to greater than greater than or equal to type comparison	left to right
== !=	is equal to is not equal to	left to right
&	bitwise AND boolean logical AND	left to right
^	bitwise exclusive OR boolean logical exclusive OR	left to right

Fig. A.1 | Operator precedence chart. (Part 1 of 2.)

Operator	Description	Associativity
|	bitwise inclusive OR boolean logical inclusive OR	left to right
&&	conditional AND	left to right
||	conditional OR	left to right
?:	conditional	right to left
= += -= *= /= %= &= ^= |= <<= >>= >>>=	assignment addition assignment subtraction assignment multiplication assignment division assignment remainder assignment bitwise AND assignment bitwise exclusive OR assignment bitwise inclusive OR assignment bitwise left-shift assignment bitwise signed-right-shift assignment bitwise unsigned-right-shift assignment	right to left

Fig. A.1 | Operator precedence chart. (Part 2 of 2.)

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Elementos de um programa Java

- *literals → representação do valor de um tipo primitivo, String ou null*
- *identifiers → nomes que identificam variáveis, classes, métodos, etc.*
- *variables → local na memória para armazenar valor de certo tipo*
- *data types → primitive type e reference type*
- *keywords → não podem ser usadas como identificadores*
- *operators → realizam operações sobre variáveis e valores (literals)*
- *separators → separam trechos de código*

- *declarations → int x;*
- *expressions → x + 5;*
- *statements → int y = x + 5*

Fonte: https://docs.oracle.com/javase/specs/jls/se21/html/index.html

Validação do ambiente

- Observar regras gerais para envio das atividades
- Implementar programa Hello, World!

1ª Atividade

- Implementar os programas
  - Calculadora de índice de massa corporal (IMC)
  - Calculadora da Área de um Polígono Regular
  - Sequência de Fibonnaci
  - Elefante visitando amigo
  - Senha forte

- Prazo: 28/02/2025 às 8h15

- **Relembrando**: 1ª leitura para o 24/02/2025 até 8h15

INSTITUTO BRASILEIRO DE ENSINO, DESENVOLVIMENTO E PESQUISA

fabricio.santana@idp.edu.br
www.linkedin.com/in/fabriciofsantana